WEB APPLICATION VULNERABILITY SCANNER

Project Report | Python • Flask • Web Application Security

Prepared by	Project domain	Project type
Keerthi	Cybersecurity	Educational security assessment tool

1. Introduction

The Web Application Vulnerability Scanner is a Python-based tool developed to perform basic, preliminary security assessments of web applications. It combines a web crawler, selected security checks, and a Flask-based interface to help users inspect web pages, HTML forms, and common security configuration indicators. The project was built as a hands-on learning exercise in Python programming, HTTP communication, web technologies, and application security.

2. Abstract

Web applications can contain weaknesses in input handling, form protection, and security configuration. This project provides an educational way to examine some of these areas. The scanner sends HTTP requests to an authorized target, parses returned HTML, discovers links and forms, and runs limited checks for reflected-input indicators, database error patterns, commonly named CSRF token fields, and selected HTTP security headers. Results are presented through a browser interface and recorded in a text report. The tool is a prototype: its observations are indicators for manual review, not proof that a vulnerability is exploitable or that an application is secure.

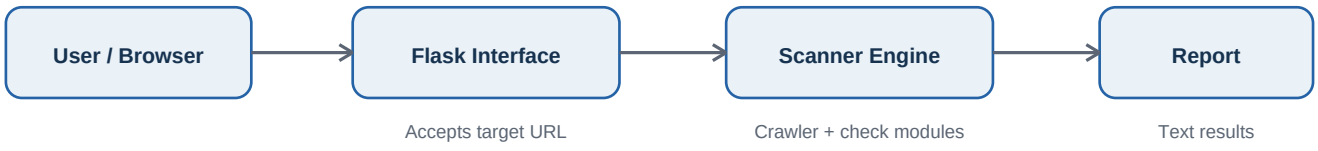
3. Tools and Technologies Used

Tool / Technology	Purpose
Python 3	Core programming language and scan orchestration.
Flask	Hosts the web interface and local test application.
Requests	Sends HTTP requests and receives responses.
BeautifulSoup	Parses HTML and extracts links, forms, and input fields.
HTML and CSS	Builds and styles the browser-based interface.
Git and GitHub	Tracks source-code changes and hosts the project repository.

4. Project Workflow / Steps Involved

- **Environment setup:** Install Python and the required packages using `requirements.txt`.
- **Target selection:** Start the local test application and scanner interface; use only a system owned by the tester or explicitly authorized for assessment.
- **Web crawling:** Send an HTTP request and parse the returned HTML to identify links, forms, form methods, actions, and input fields.
- **Security checks:** Run limited checks for reflected input, common SQL error strings, commonly named CSRF token fields, and selected HTTP response headers.
- **Reporting:** Collect observations and supporting details in the scan output and display them through the Flask interface.
- **Validation:** Review each observation manually and treat it as a lead for further testing rather than a confirmed vulnerability.

5. Project Architecture



6. Conclusion

The project demonstrates how Python libraries and Flask can be combined to build a simple web application security assessment workflow. It provided practical experience with HTTP requests, HTML parsing, form discovery, basic security indicators, report generation, and GitHub-based version control. The current implementation has limited coverage and should not be considered a replacement for professional security testing. Future improvements could include stronger input validation,

automated tests, richer reports, safer scan controls, and additional security checks. All testing should be performed only with proper authorization.

Project repository: https://github.com/keerthi-project/Web-Application-Vulnerability-Scanner
Note: Findings such as reflected input, SQL error patterns, or a missing commonly named CSRF field are preliminary indicators and require manual verification.